



Full Technical Documentation

aws-iam-monitor

Real-time AWS IAM Auditing, Security Monitoring and Automated Threat
Detection Platform

Version: 1.2.0
Author: ZouariOmar
License: Apache-2.0
Repository: <https://github.com/ZouariOmar/aws-iam-monitor>
Contact: zouariomar20@gmail.com

This document consolidates the project's reference documentation – overview, architecture, installation, usage and security – into a single, self-contained technical reference for the `aws-iam-monitor` v1.2.1 release.

Contents

1	Overview	1
1.1	Introduction	1
1.2	Key Features	1
1.3	Repository Structure	1
1.4	Documentation Map	2
2	Architecture	3
2.1	System Data Flow	3
2.2	AWS Services & Responsibilities	3
2.3	Deployment Ordering	4
2.4	Terraform vs. Bash	5
2.5	Cost Estimate	5
3	Installation	7
3.1	Clone Repository	7
3.2	Requirements	7
3.2.1	Bash Implementation	7
3.2.2	Terraform Implementation	7
3.2.3	Documentation Build (optional)	7
3.3	Setup	8
3.3.1	Bash	8
3.3.2	Terraform	8
3.3.3	Docker	8
4	Usage	9
4.1	Makefile	9
4.1.1	Main	9
4.1.2	Bash Implementation	9
4.1.3	Terraform Implementation	10
4.1.4	Documentation	10
4.1.5	Docker	10
4.2	Bash – <code>awsctl</code>	10
4.2.1	Command Syntax	10
4.2.2	Actions	11
4.2.3	Targets	11
4.2.4	Options	11
4.2.5	Deployment Examples	11
4.3	Terraform	12
4.3.1	Command Syntax	12
4.3.2	Deployment Examples	12
5	Security	13
5.1	Supported Versions	13
5.2	Reporting a Vulnerability	13
5.2.1	How to Report	13

5.2.2	Response Process	13
5.3	Security Practices	13
5.3.1	Least Privilege	13
5.3.2	IP Whitelisting	14
5.3.3	Idempotent, Safe Operations	14
5.3.4	Secure AWS Integrations	14
5.3.5	Data Protection	14
5.3.6	Secrets Management	14
5.3.7	Logging	14
5.3.8	Dependency Management	14
5.4	Security Contributions	15
5.5	AWS Security Recommendations	15
5.6	Resources	15
6	Results & Illustrations	16
6.1	SNS Subscription Confirmation	16
6.2	High-Risk Alerts	16
6.3	Unauthorized IP Access Detection	16
7	Release Notes	19
7.1	v1.2.1 – Current Release	19
7.2	v1.1.0	19
7.3	v1.0.0	19
	License & Contact	20
	License	20
	Contact & Support	20

Chapter 1

Overview

1.1 Introduction

aws-iam-monitor is a production-ready, serverless AWS security solution designed to provide continuous, real-time visibility into Identity and Access Management (IAM) changes across AWS accounts and AWS Organizations.

By capturing management events from **AWS CloudTrail**, filtering events with **Amazon EventBridge**, and processing security payloads with **AWS Lambda**, the system detects unauthorized modifications, privilege escalation risks, new credential creation, and policy tampering within seconds. High-risk events immediately trigger **Amazon SNS** notifications (with optional automated email subscriptions) while maintaining structured audit logs in **Amazon S3** and custom performance metrics in **Amazon CloudWatch**.

1.2 Key Features

- **Real-time Event Detection:** Instant detection of sensitive IAM operations (e.g. `CreateUser`, `DeletePolicy`, `CreateAccessKey`, `DeactivateMFADevice`, `AttachUserPolicy`).
- **IP Whitelisting & Enforcement:** Automated generation of `IPWhitelistPolicy` and real-time source IP validation within the monitoring Lambda.
- **Risk Classification:** Automated risk grading (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) for every IAM API call.
- **Dedicated SNS Management Component:** Dedicated `sns` module with automated email alert subscriptions via `.env` or CLI parameters.
- **Audit Trail Archiving:** Every IAM event is saved as a structured JSON record in Amazon S3 grouped by date (`YYYY/MM/DD/`).
- **CloudWatch Metrics:** Emits custom `AWSIAMMonitor` metrics for CloudWatch dashboards and alarm creation.
- **Idempotent Infrastructure:** Both the Bash CLI (`awsctl`) and the Terraform implementation create, update, and delete resources safely without duplication.
- **Two Equivalent Implementations:** An imperative Bash/AWS-CLI CLI and a declarative Terraform implementation, pick whichever fits your workflow.

1.3 Repository Structure

```
aws-iam-monitor/  
|-- Makefile  
|-- README.md  
|-- ARCHITECTURE.md  
|-- INSTALL.md  
|-- USAGE.md  
|-- SECURITY.md
```

```

|-- SUPPORT.md
|-- project/
| |-- bash/ # Bash / AWS CLI implementation
| | |-- .env.example # Environment configuration template
| | |-- awsctl # Primary orchestration CLI binary
| | |-- lib/ # Shared helper library
| | |-- iam/ # IAM module
| | |-- cloud-trail/ # CloudTrail module
| | |-- sns/ # SNS module
| | |-- event-bridge/ # EventBridge module
| | |-- lambda/ # Lambda module
| | |-- Makefile # Bash implementation Makefile
| | '-- README.md # Bash implementation reference
|-- terraform/ # Terraform implementation
|-- main.tf # Module wiring
|-- variables.tf # Input variables
|-- outputs.tf # Output values
|-- providers.tf # AWS provider configuration
|-- versions.tf # Terraform & provider version constraints
|-- locals.tf # Computed values
|-- terraform.tfvars.example
|-- modules/ # iam, cloudtrail, sns, lambda,
| # eventbridge, s3, cloudwatch
|-- Makefile # Terraform implementation Makefile
|-- README.md # Terraform implementation reference
-- docapi/ # Sphinx documentation site source
'-- res/ # Documentation assets & logos

```

1.4 Documentation Map

Document	Covers
README.md	Project overview, key features, quickstart.
ARCHITECTURE.md	Architecture diagram, AWS services table, deployment ordering.
INSTALL.md	Prerequisites, requirements, and setup for both implementations.
USAGE.md	Full Bash (<code>awsctl</code>) and Terraform CLI/command reference, deployment examples.
SECURITY.md	Security policy, vulnerability reporting, security best practices.
project/bash/README.md	Bash implementation deep reference.
project/terraform/README.md	Terraform implementation deep reference, including differences from Bash.
docapi (Sphinx site)	Full documentation site source, built with <code>make docs</code> .

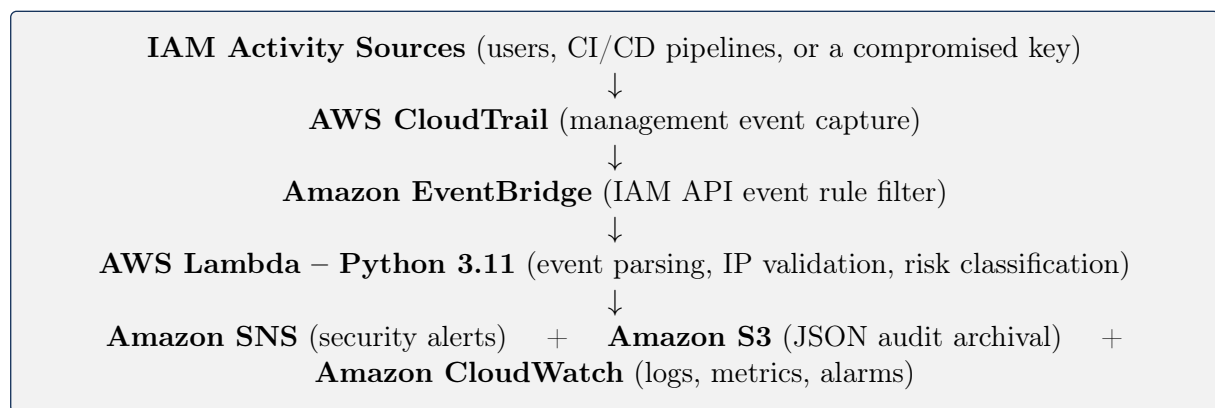
Chapter 2

Architecture

This architecture is provisioned identically by both implementations; see Chapter 4 to deploy it via either `project/bash/` (imperative) or `project/terraform/` (declarative).

2.1 System Data Flow

The architecture follows an event-driven serverless pattern:



2.2 AWS Services & Responsibilities

AWS Service	Purpose
AWS IAM	Customer-managed policies, groups, users, execution roles, and trust policies.
AWS CloudTrail	Captures AWS account management events and API calls across the account.
Amazon EventBridge	Evaluates CloudTrail events against IAM pattern filters and invokes Lambda.
AWS Lambda	Python 3.11 handler: source IP validation, risk classification, S3 audit logging, SNS alert publishing, CloudWatch metric emission.
Amazon SNS	Topic, topic policy, and email subscription management for security alerts.
Amazon S3	Server-side encrypted (AES-256), versioned storage for CloudTrail logs and historical IAM audit records.
Amazon CloudWatch	Log group retention management, custom metrics (AWSIAMMonitor), and alarms.

2.3 Deployment Ordering

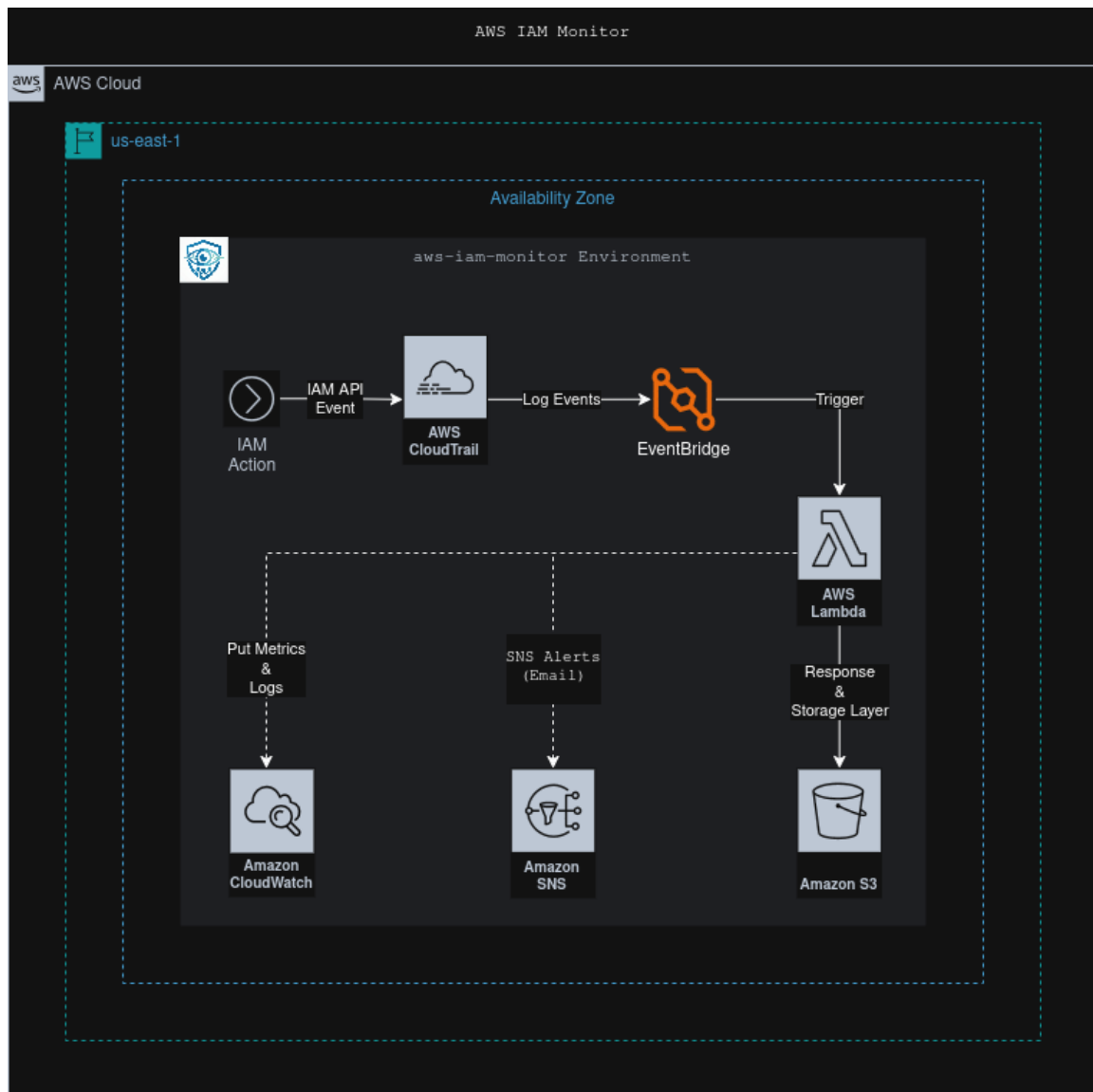


Figure 2.1: Full architecture diagram of `aws-iam-monitor`, identical across both the Bash and Terraform implementations.

The Bash implementation provisions this architecture as an explicit 5-stage sequence (`./awsctl -up all`); the Terraform implementation resolves the same ordering automatically from its module dependency graph (`terraform apply`); both converge on the same end state.

1. **IAM**: Customer-managed policies (`DevPolicy`, `ProdPolicy`, `AuditPolicy`), groups, users, and optional execution roles.
2. **CloudTrail**: S3 log bucket and CloudTrail management events trail.
3. **SNS**: Alert topic (`iam-alerts`), topic policy, and email subscription.
4. **Lambda**: S3 audit history bucket, execution role, packaged `lambda_function.py`, and CloudWatch log group.
5. **EventBridge**: Rule pattern registration, Lambda target attachment, and invocation permission.

Teardown (`./awsctl -down all` / `terraform destroy`) reverses this order.

2.4 Terraform vs. Bash

Both implementations provision the same architecture described above; they are independent, equivalent implementations, not a sequential pipeline versus a functionally different declarative one. Terraform additionally provisions optional CloudWatch alarms on the metrics Lambda already emits (`UnauthorizedIPAccess` and function `Errors`), which is a Terraform-only enhancement, not a divergence in required behavior.

Aspect	Bash / AWS CLI	Terraform
Approach	Imperative (explicit command sequence)	Declarative (target state described, order resolved automatically)
Lambda IAM permissions	Broad scope (<code>S3AuditHistory</code> , <code>Resource: "*" for SNS</code>)	Strictly scoped to exact resources (reinforced least privilege)
S3 public access	Not explicitly blocked	Explicitly blocked on both buckets
IAM propagation wait	6-attempt retry loop with a 10s delay	Deterministic single wait (<code>time_sleep, 10s</code>)
Lambda code deployment	Manual SHA-256 hash comparison	Automatic via <code>source_code_hash</code>
CloudWatch alarms	Absent (metrics emitted but unused)	Optional, enabled by default
Reproducibility	High, dependent on script execution order	High, guaranteed by the dependency graph
Maintenance	Simple for small, targeted changes	Facilitated by modularity and declarative state for complex changes

Note

See `project/terraform/README.md` (section “Differences from the Bash implementation”) for the complete comparison. Outside these assumed differences, resource naming, the EventBridge event pattern, the IAM policy documents (`DevPolicy`, `ProdPolicy`, `AuditPolicy`, `IPWhitelistPolicy`) and the SNS topic policy remain strictly identical between both implementations.

2.5 Cost Estimate

Both implementations provision the same resources and therefore share the same cost profile. The figures below are modeled with the AWS Pricing Calculator (calculator.aws), Region US East (N. Virginia), at standard on-demand rates with the AWS Free Tier *not* applied, for a baseline of roughly 50,000 IAM management events per month.

AWS service	Basis	Est./mo
AWS CloudTrail	50k IAM management events @ \$2.00 / 100k	\$1.00
Amazon EventBridge	AWS service events on the default bus (no charge)	\$0.00
AWS Lambda	50k requests, 128 MB, ~400 ms (~2,500 GB-s)	\$0.05
Amazon SNS	50k publishes + ~500 email deliveries	\$0.04
Amazon S3	~10 GB storage, ~55k PUT/GET	\$0.51
Amazon CloudWatch	3 custom metrics, 2 alarms, 50k <code>PutMetricData</code> , ~1 GB logs	\$2.13
Data transfer	Intra-region + minimal internet egress	\$0.09
Estimated total		\$3.82

That is approximately \$3.82 per month (~\$46 per year). Cost scales almost entirely with the IAM event volume: a light workload (5k events/mo) costs about \$2, a moderate one (50k) about \$4, and a heavy one (500k) about \$24 per month.

Note

Amazon CloudWatch dominates the bill (custom metrics plus one `PutMetricData` call per event); batching metric publishes reduces it further. Within the AWS Free Tier most line items fall to \$0 and the effective total is negligible.

Chapter 3

Installation

aws-iam-monitor ships two independent, equivalent infrastructure implementations; install prerequisites for whichever you plan to use (or both). See Chapter 4 once installed, for the CLI reference of both.

3.1 Clone Repository

```
git clone https://github.com/ZouariOmar/aws-iam-monitor.git
cd aws-iam-monitor
```

3.2 Requirements

- Git
- An AWS Account with access to: CloudTrail, EventBridge, Lambda, STS AssumeRole, SNS, S3, CloudWatch
- Make (optional, but recommended, wraps both implementations)

3.2.1 Bash Implementation

- **Operating System:** Linux / macOS
- **Shell:** Bash v5.0+
- **aws-cli** (v2.0+)
- **jq** (v1.6+)
- **sed** (v4.10+)
- **zip** (v3.0+)
- **openssl** (v3.6+)
- **python3** (v3.11+)

3.2.2 Terraform Implementation

- Terraform $\geq$ 1.5.0
- The same AWS account access as above
- Network access to the Terraform Registry (**terraform init** downloads the **hashicorp/aws**, **hashicorp/archive**, and **hashicorp/time** providers automatically, no other local tooling required)

3.2.3 Documentation Build (optional)

Python 3.14+ and uv (for the Sphinx docapi site, see **make docs**).

3.3 Setup

3.3.1 Bash

```
cd project/bash

# Configure environment variables (optional: SNS email, IP whitelist)
cp .env.example .env
nano .env # SNS_ALERT_EMAIL=..., ALLOWED_IPS=1.2.3.4/32,5.6.7.8/24

# Ensure CLI executable permissions (or: make -C project/bash chmod)
chmod +x awsctl \
  lib/* \
  iam/src/* \
  cloud-trail/src/* \
  sns/src/* sns/test/* \
  event-bridge/src/* event-bridge/test/* \
  lambda/src/* lambda/test/*

# Verify AWS CLI authentication
aws sts get-caller-identity
```

3.3.2 Terraform

```
cd project/terraform
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars # sns_alert_email, allowed_ips, etc.
terraform init
```

See `project/terraform/README.md` for the full reference.

3.3.3 Docker

Each implementation has its own Dockerfile (`project/bash/Dockerfile`, `project/terraform/Dockerfile`) and its own set of Makefile targets, prefixed `bash-docker-*` and `tr-docker-*` respectively:

```
make bash-docker # build + run + open a shell (Bash implementation)
make tr-docker # build + run + open a shell (Terraform implementation)
```

Chapter 4

Usage

aws-iam-monitor ships two independent, equivalent infrastructure implementations, pick whichever fits your workflow. Do not run both against the same AWS account simultaneously; they use the same default resource names and would conflict.

Important

See Chapter 4 above if you haven't set up prerequisites yet (Installation chapter).

4.1 Makefile

The root Makefile delegates Bash- and Terraform-specific commands to their own sub-Makefiles (`project/bash/Makefile`, `project/terraform/Makefile`), which remain independently usable via `make -C project/bash <target>` / `make -C project/terraform <target>`. Run `make help` at any level for the full, self-documenting list.

4.1.1 Main

Target	Description
<code>make init</code>	Init the project (<code>uv venv</code>).
<code>make sync</code>	Sync dependencies (<code>uv sync</code>).
<code>make run</code>	Run the Bash implementation (<code>ARGS=...</code>).
<code>make test</code>	Run the Bash implementation's tests.
<code>make clean</code>	Clean the project.
<code>make changelog</code>	Generate <code>CHANGELOG.md</code> .
<code>make version-up</code>	Bump project version and create a git tag.
<code>make gh-issue</code>	Create a GitHub issue from a template (<code>TITLE=...</code> <code>FILE=...</code>).

4.1.2 Bash Implementation

Target	Description
<code>make bash-up TARGET=...</code>	Provision infrastructure (<code>TARGET</code> default: <code>all</code>).
<code>make bash-down TARGET=...</code>	Tear down infrastructure.
<code>make bash-status TARGET=...</code>	Show infrastructure status.
<code>make bash-test TARGET=...</code>	Run the Bash test suite.

4.1.3 Terraform Implementation

Target	Description
<code>make tf-init</code>	<code>terraform init.</code>
<code>make tf-fmt</code>	<code>terraform fmt.</code>
<code>make tf-validate</code>	<code>terraform validate.</code>
<code>make tf-plan</code>	<code>terraform plan.</code>
<code>make tf-apply</code>	<code>terraform apply.</code>
<code>make tf-destroy</code>	<code>terraform destroy.</code>
<code>make tf-output</code>	<code>terraform output.</code>

4.1.4 Documentation

Target	Description
<code>make docs</code>	Build the Sphinx docs to HTML.
<code>make docs-serve</code>	Serve the built docs locally.
<code>make docs-preview</code>	Build and preview docs (Vercel).
<code>make docs-deploy</code>	Build and deploy docs (Vercel).

4.1.5 Docker

Target	Description
<code>make bash-docker</code>	Build + run + open a shell (Bash).
<code>make bash-docker-build</code>	Build the Bash Docker image.
<code>make bash-docker-bash</code>	Open a shell inside the Bash container.
<code>make bash-docker-run</code>	Run <code>awsctl</code> inside the Bash container.
<code>make bash-docker-stop</code>	Stop the running Bash container.
<code>make bash-docker-clean</code>	Remove the Bash Docker image.
<code>make tr-docker</code>	Build + run + open a shell (Terraform).
<code>make tr-docker-build</code>	Build the Terraform Docker image.
<code>make tr-docker-bash</code>	Open a shell inside the Terraform container.
<code>make tr-docker-run</code>	Run <code>terraform</code> inside the Terraform container.
<code>make tr-docker-stop</code>	Stop the running Terraform container.
<code>make tr-docker-clean</code>	Remove the Terraform Docker image.

4.2 Bash – awsctl

The primary orchestration tool is `./awsctl`, run from `project/bash/`. It provides a unified command line interface for provisioning, managing, testing, and destroying infrastructure components.

4.2.1 Command Syntax

```
cd project/bash
./awsctl <ACTION> [TARGET] [OPTIONS]
```

4.2.2 Actions

- **-up** : Create/configure resources.
- **-down** : Tear down/delete resources.
- **-status** : Check status of resources.
- **-test** : Execute test suites.

4.2.3 Targets

- **all** (*default*) : Complete monitoring infrastructure.
- **iam** : IAM policies, groups, users, and roles.
- **cloud-trail** : CloudTrail trail and log S3 bucket.
- **sns** : SNS security alert topic and email subscriptions.
- **lambda** : Lambda function and audit S3 bucket.
- **event-bridge** : EventBridge rule and target registration.

4.2.4 Options

- **-r, -role** : Include optional IAM roles during IAM setup.
- **-v, -verbose** : Display detailed AWS CLI execution logs.
- **-h, -help** : Show help message.

4.2.5 Deployment Examples

```
./awsctl --up all # Provision the complete monitoring pipeline
```

This will automatically:

1. Create customer-managed IAM policies (**DevPolicy**, **ProdPolicy**, **AuditPolicy**), groups (**DevGroup**, **ProdGroup**, **TestGroup**), and users (**DevAdmin**, **ProdAdmin**, etc.).
2. Provision an S3 log bucket and create a CloudTrail management events trail.
3. Provision the SNS alert topic (**iam-alerts**) and subscribe **SNS_ALERT_EMAIL** if configured in **.env**.
4. Provision an S3 audit history bucket and deploy the monitoring Lambda function with its execution role.
5. Create an EventBridge rule for IAM events, attach the Lambda function as a target, and grant invocation permissions.

```
./awsctl --up sns # Provision only the SNS alert component
./awsctl --status all # Check status of every component
./awsctl --test all # Run automated test suites
./awsctl --down all # Tear down the complete infrastructure
```

Note

For prerequisites, setup, module structure, and a detailed comparison against the Terraform implementation, see `project/bash/README.md`.

4.3 Terraform

The equivalent declarative implementation lives under `project/terraform/`.

4.3.1 Command Syntax

```
cd project/terraform
terraform init
terraform plan
terraform apply
terraform destroy
```

A plain `terraform apply` with default variables provisions the same architecture as `./awsctl -up all`, including the demo IAM sandbox (`enable_iam_sandbox = true` by default). Configuration is entirely variable-driven via `terraform.tfvars` (copy from `terraform.tfvars.example`) – Terraform does not read the Bash implementation’s `.env` file.

4.3.2 Deployment Examples

```
# First-time provisioning
cp terraform.tfvars.example terraform.tfvars
terraform init
terraform apply

# Updating infrastructure, edit terraform.tfvars, then:
terraform plan
terraform apply

# Teardown
terraform destroy
```

Changing the shared Lambda source (`project/bash/lambda/src/lambda_function.py`) is detected automatically via `source_code_hash` and redeployed on the next `apply`, no manual packaging step.

Note

For the full variable reference, state security guidance, and a detailed comparison against the Bash implementation, see `project/terraform/README.md`.

Chapter 5

Security

5.1 Supported Versions

The following versions of **aws-iam-monitor** receive security updates.

Version	Supported
1.x.x	yes
0.x.x	no

5.2 Reporting a Vulnerability

If you discover a security vulnerability in **aws-iam-monitor**, please report it responsibly.

5.2.1 How to Report

Send an email to **zouariomar20@gmail.com** containing:

- A description of the vulnerability
- Steps to reproduce
- The affected version
- Any relevant logs or screenshots
- A proof of concept (if available)

Please **do not open a public GitHub issue** for security vulnerabilities.

5.2.2 Response Process

We aim to:

- Acknowledge your report within **48 hours**
- Investigate and validate the issue
- Keep you informed throughout the remediation process
- Release a security fix as quickly as practical
- Credit you for the disclosure (if desired)

5.3 Security Practices

The project follows security best practices for AWS environments.

5.3.1 Least Privilege

- IAM policies grant only the permissions required.

- Cross-account access should use read-only roles whenever possible.
- AWS STS AssumeRole is recommended for organization-wide deployments.

5.3.2 IP Whitelisting

- The system supports automated `IPWhitelistPolicy` generation to restrict IAM actions to trusted networks.
- Continuous monitoring validates the source IP of every IAM event against a pre-configured `ALLOWED_IPS` list.
- Unauthorized access attempts from blocked IPs trigger high-priority alerts and custom CloudWatch metrics.

5.3.3 Idempotent, Safe Operations

- Both implementations check resource state before creating, updating, or deleting it, to avoid duplicate resources and unintended service disruptions.
- The Bash implementation's `awsctl` and its `*_ctl` sub-scripts verify existence/hashes before acting; the Terraform implementation relies on Terraform's own state-diffing (`terraform plan`) for the same guarantee, a second `terraform plan` after `apply` should show no changes.

5.3.4 Secure AWS Integrations

aws-iam-monitor relies on AWS managed services including AWS CloudTrail, Amazon EventBridge, AWS Lambda, Amazon SNS, Amazon S3, Amazon CloudWatch, and AWS STS. These services should be configured according to AWS security best practices.

5.3.5 Data Protection

- Communication with AWS APIs uses HTTPS/TLS.
- Audit reports stored in Amazon S3 should use encryption at rest.
- Sensitive configuration should never be committed to the repository.

5.3.6 Secrets Management

Never commit: AWS Access Keys, Secret Access Keys, Session Tokens, `.env` files, private certificates, or customer data.

Use one of the following instead: IAM Roles, AWS Secrets Manager, AWS Systems Manager Parameter Store, or environment variables.

5.3.7 Logging

aws-iam-monitor records IAM activity for auditing purposes. Logs should exclude secrets and credentials, be retained according to organizational policies, and be monitored for suspicious activity.

5.3.8 Dependency Management

Dependencies should be kept up to date. Recommended tools include Dependabot, `pip-audit`, `uv`, and GitHub Dependabot Alerts.

5.4 Security Contributions

Contributors can help improve security by following secure coding practices, validating all external input, avoiding hard-coded credentials, using least-privilege IAM permissions, reviewing dependency updates, and testing security-sensitive changes before submitting pull requests.

5.5 AWS Security Recommendations

For production deployments, we recommend:

- Enable AWS CloudTrail in all regions.
- Enable S3 bucket encryption.
- Enable CloudWatch log retention.
- Use SNS encryption where appropriate.
- Enable MFA for privileged AWS accounts.
- Rotate IAM credentials regularly.
- Use dedicated monitoring accounts within AWS Organizations.

5.6 Resources

- AWS Well-Architected Framework – Security Pillar
- AWS IAM Best Practices
- AWS Security Best Practices
- OWASP Secure Coding Practices Checklist
- CWE Top 25 Most Dangerous Software Weaknesses

Chapter 6

Results & Illustrations

The following screenshots, captured on the AWS test account used during development, illustrate the system's effective behavior once deployed: an IAM event is captured by CloudTrail, filtered by the EventBridge rule, analyzed and classified by the Lambda function, archived to the audit bucket, and – for high-risk events – notified via the SNS topic `iam-alerts`.

6.1 SNS Subscription Confirmation



Figure 6.1: Confirmation e-mail received when subscribing to the `iam-alerts` SNS topic.

6.2 High-Risk Alerts

6.3 Unauthorized IP Access Detection

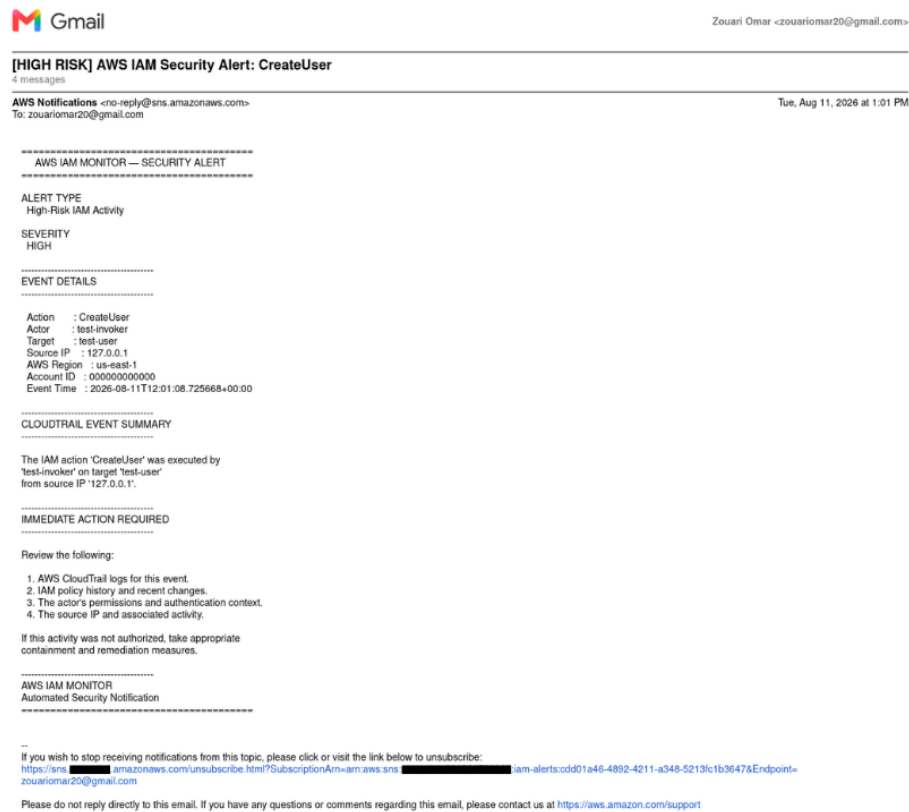


Figure 6.2: Alert notification for an event classified as high risk: IAM user creation (CreateUser).

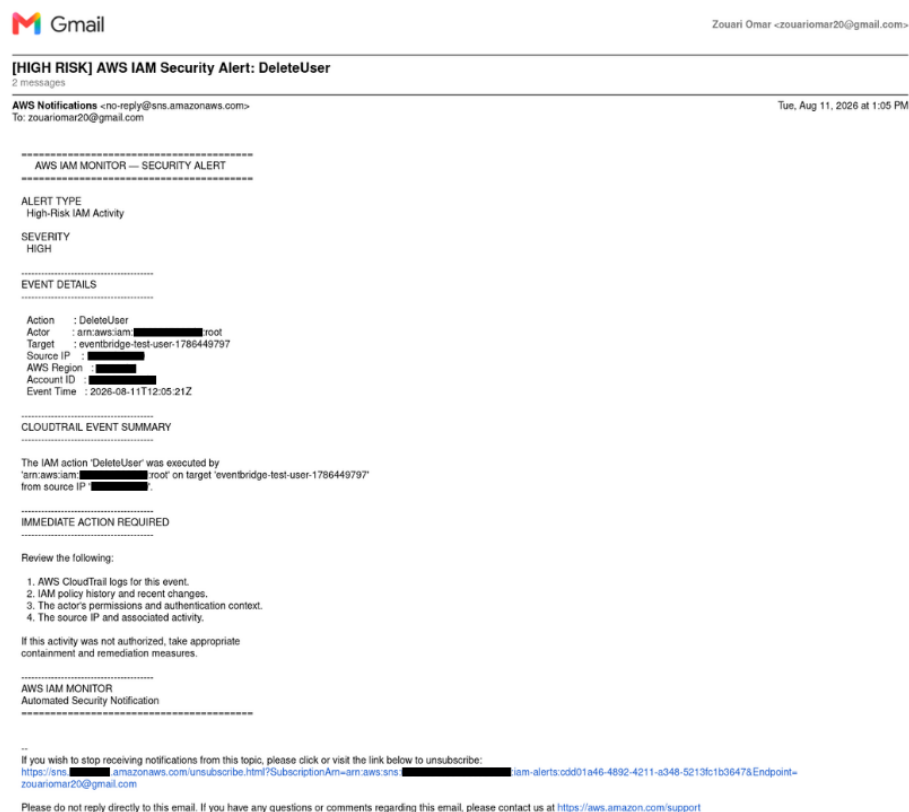


Figure 6.3: Alert notification for an event classified as high risk: IAM user deletion (DeleteUser).

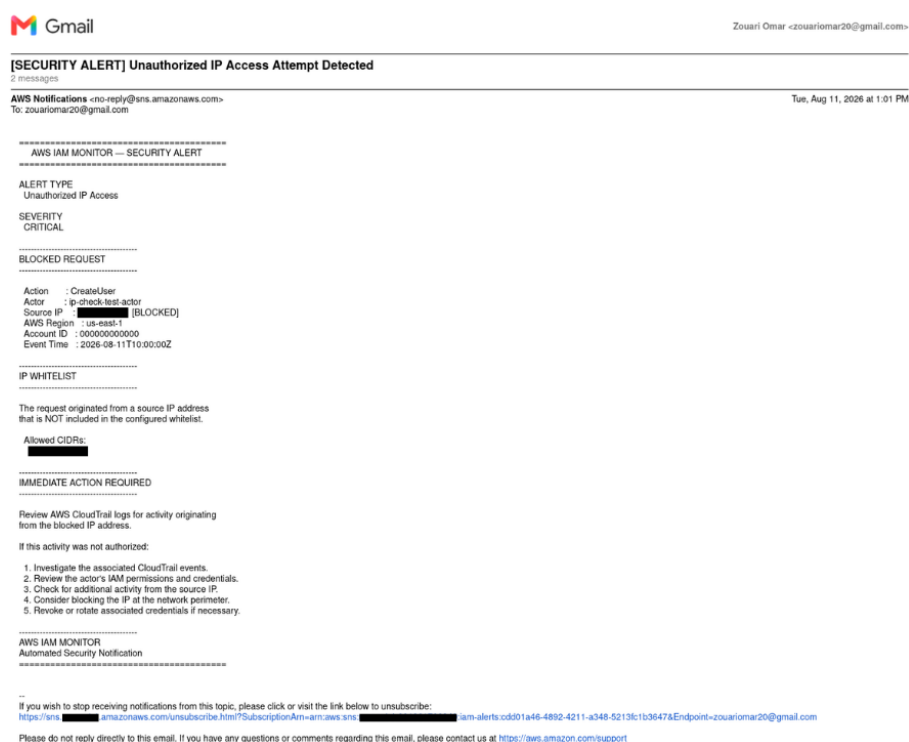


Figure 6.4: Alert triggered when an IAM operation originates from a source IP absent from the configured whitelist.

Chapter 7

Release Notes

7.1 v1.2.1 – Current Release

- **Features:** Makefile GitHub issue target (#13); Terraform AWS infrastructure implementation.
- **Other:** Support for Bash submodule in the root build.
- **Refactor:** Bash tooling moved into its own subdirectory.
- **Documentation:** v1.1.0 changelog; updated architecture diagram; Terraform documentation added to docapi; project documentation updates.

7.2 v1.1.0

- **Features:** Lambda source IP whitelist enforcement; IAM whitelist policy provisioning; improved Docker workflow and Makefile.
- **Bug Fixes:** Makefile version handling; SNS subscription state handling; CloudTrail S3 status checks region fix.
- **Refactor:** Modernized Lambda timestamps and security alerts.
- **Documentation:** Security documentation update.

7.3 v1.0.0

- **Features:** EventBridge IAM CloudTrail event pattern policy and rule management controller – first stable release of the complete monitoring pipeline (IAM, CloudTrail, EventBridge, Lambda, SNS, S3, CloudWatch).

Note

See `CHANGELOG.md` at the repository root for the complete, auto-generated release history (generated with `make changelog`).

License & Contact

License

This project is licensed under the **Apache-2.0 License**. See the `LICENSE` file at the repository root for details.

Contact & Support

- **Author:** ZouariOmar (<https://github.com/ZouariOmar>)
- **Email:** zouariomar20@gmail.com
- **LinkedIn:** <https://www.linkedin.com/in/zouari-omar>
- **Issues:** <https://github.com/ZouariOmar/aws-iam-monitor/issues>
- **Support:** See `SUPPORT.md`